LAN Whiteboard

Computer Networks Lab Project

Bupesh - CS24B2029
Rakesh - CS24B2039
Suhas - CS24B2040
Guna - CS24B2043

1. Aim

Build a shared whiteboard that works on a LAN without internet. Users should be able to draw at the same time, send chat messages and continue from the same board after joining or reconnecting. The application uses a central server and TCP sockets.

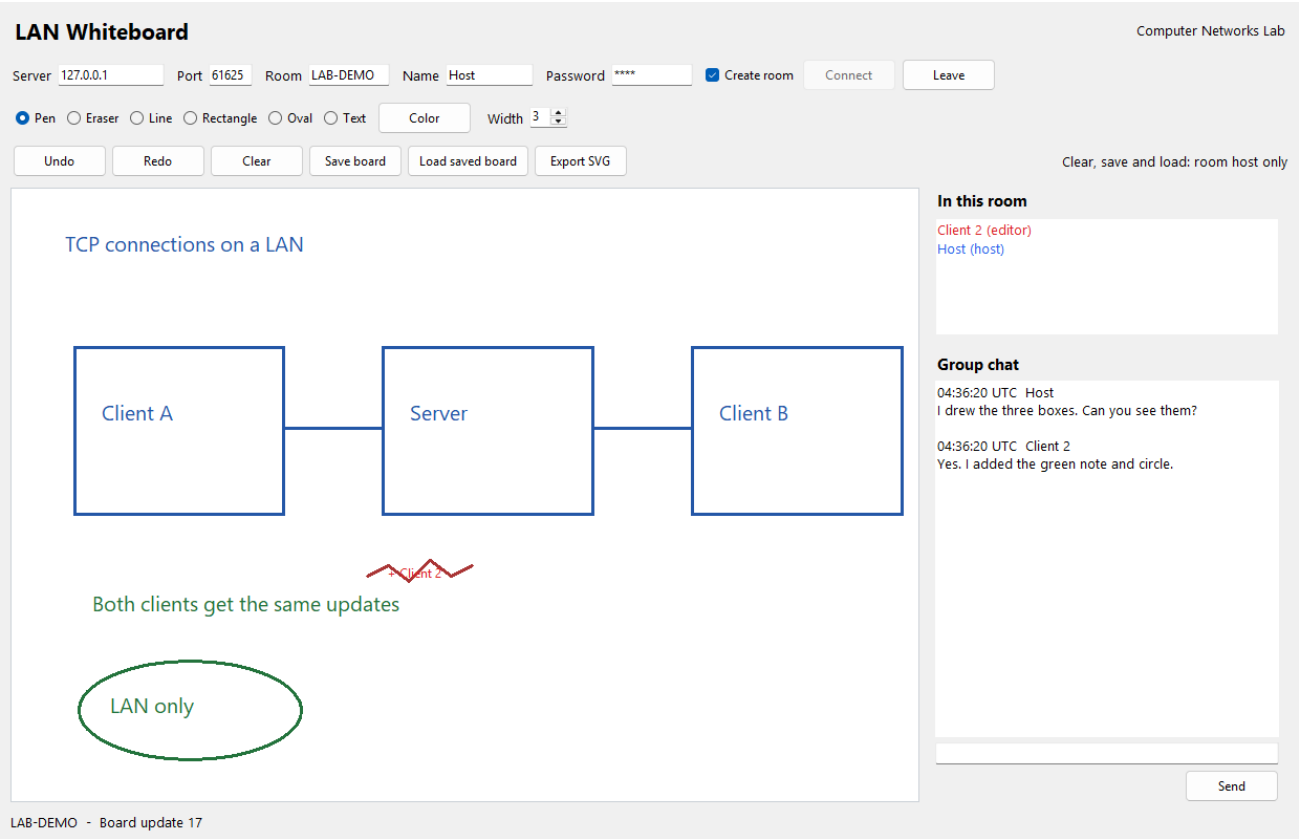


Figure 1. Host window from a local test with two desktop clients. The second client added the green note and circle.

What the application includes

The canvas supports pen, eraser, lines, rectangles, ovals and text. Users can choose a color and width, undo their own strokes, export the board and chat. The room host can save, clear and load a shared board. New clients receive the existing board when they join.

2. Network design

One machine runs server.py. Each client opens a TCP connection to that machine. The server keeps a separate board and chat history for each room. A room code and password identify the session.

Why the server keeps the board

If two clients draw together, they need to apply those edits in the same order. The server holds a lock while it checks an action, changes the room state and gives the action an update number. It then puts the update in each participant's outgoing queue. A separate writer thread sends that queue over the socket, so a slow connection does not hold up a socket write inside the state lock.

Message format

Part	Contents
First 4 bytes	Unsigned big-endian integer N: the number of payload bytes
Next N bytes	UTF-8 JSON object. Maximum payload size: 8 MiB

TCP carries a stream of bytes. It may split one JSON message across several recv() calls or deliver several messages together. protocol.exact() keeps reading until it has the requested number of bytes. The receiver reads the header first, then exactly N payload bytes. Invalid lengths, invalid JSON and closed connections end that client connection.

Message	Use
join	Room code, username, password, create flag and optional recovery token
welcome / state	Client identity, role, current update number, drawings, chat and participants
draw	Request ID and drawing tool, points, color, width, text and stroke group
chat	Request ID and text. The server adds the sender name and UTC time.
undo / redo	Change the active state of one user's stroke group
save / clear / restore	Host controls for the shared board
event / ack / error	Accepted update, duplicate acknowledgement or rejected request
sync / presence / cursor	Full-state request, participant changes and temporary cursor updates
ping / pong	Connection heartbeat

Example drawing request

{"type":"draw","request":"a1","op":{"tool":"pen","points":[[100,200],[150,240]],"color":"#2457a7","width":3,"group":"drag1"}}

Coordinates range from 0 to 10000 in each direction. The client converts them to its current canvas size. The server supplies the sender identity rather than accepting a drawing owner from the message.

3. Board history and recovery

Drawing and undo

During a pen or eraser drag, the client sends points in small chunks. All chunks from the same drag share a group ID. The server stores operations in order, including their tool, points, color, width and active flag. Shapes commit when the mouse is released. Text commits after the annotation is entered.

Undo disables the latest active group belonging to that user. Redo enables their most recently undone group. Drawing again drops that user's redo branch. Another user's drawing stays in the history. Clear removes the current board and its undo history. The eraser works as a white stroke, so replaying the ordered operations also defines what happens when drawings overlap.

Joining and lost connections

The server queues the initial board copy while holding the same lock used for new edits. That prevents a drawing from slipping between the initial copy and later updates. Each client expects consecutive update numbers. If it sees a gap, it requests the full board again.

Clients send a heartbeat every 5 seconds. The server's read timeout is 20 seconds. A dropped client pauses editing and retries every 2 seconds. It receives a fresh board before sending unacknowledged actions again. The server remembers the last 20,000 accepted request IDs per identity, so ordinary retries do not add the same action twice. This history is bounded.

Saving the session

After an accepted change, the server writes the rooms to a temporary JSON file and replaces sessions.json. The file holds the drawings, recent chat, saved boards, user identities and request history. Restarting the server with the same file recovers those rooms. There is one saved board per room. Save board replaces the previous saved copy, and Load saved board replaces the current canvas for everyone.

The client stores its recovery token in .cn_whiteboard_tokens.json in the user's home folder. Keeping that file lets the same user recover their identity and host permissions. Moving to another computer without the token creates a new identity.

Permissions

The first participant who creates the room becomes its host. Only that identity can save, clear or load the shared board. The server checks the permission, even if a client tries sending the command directly. Passwords use salted PBKDF2-HMAC-SHA256 hashes with 100,000 iterations in the state file. Tokens are random bearer secrets.

The server also checks tools, color strings, widths, point counts, coordinates, text lengths and frame sizes. Room passwords and board contents travel without TLS. The application is intended for a trusted LAN.

4. Running the application

Windows EXEs

1. Extract the submission ZIP to a writable folder.
2. Run release/WhiteboardServer.exe. The default TCP port is 5050.
3. On that computer, open WhiteboardClient.exe and use server address 127.0.0.1. Enter a room, name and password, tick Create room and connect.
4. Other computers use the server's LAN IPv4 address and the same port, room and password. They leave Create room unticked.

Use ipconfig to find the server address. Allow the server through Windows Firewall on the private network. Guest Wi-Fi isolation may prevent devices from reaching each other. Internet can be off while the LAN stays active.

Controls

Control	Behavior
Pen / Eraser / shapes / Text	Draw on the shared canvas. Color and Width apply to new drawings.
Undo / Redo	Change your own last stroke group
Save board	Host saves one shared copy on the server
Clear / Load saved board	Host replaces the board for everyone. A confirmation appears first.
Export SVG	Save a local image that opens in a browser offline
Leave	Stop the client connection. Other participants continue.

Source and build commands

Run the server and client in separate terminals:
python server.py
python client.py

Run tests with python -m unittest -v. Python 3.10+ with Tk is enough for the application. To build the Windows EXEs, install PyInstaller and run build.ps1. The EXEs include Python and Tk, so the demo machines do not need Python installed.

If a connection fails

Check that the server is running, both sides use the same port, and the remote client uses the LAN address rather than 127.0.0.1. If joining times out, check the firewall and Wi-Fi isolation. If host controls are rejected, reconnect using the original host identity. Keep sessions.json in a writable folder and do not edit it while the server runs.

Lab submission

Include the server and client EXEs, source, report, slides and explanation video in one ZIP. Each member presents their own contribution with name and face visible in the video. A live app demo is not required in the recording. All members attend the separate in-person demo. The assignment deadline is 15 November 2026, 11:55 pm IST.

5. Tests and screenshots

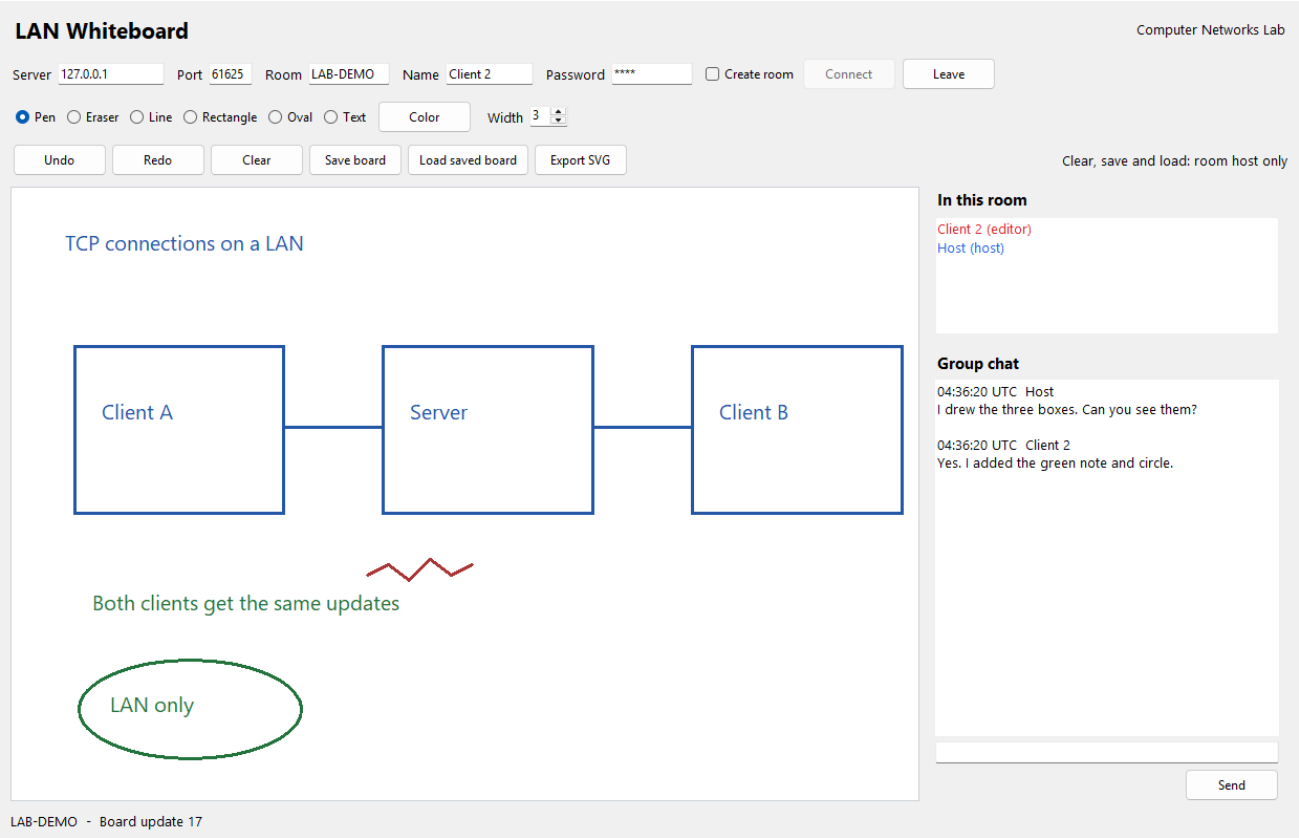


Figure 2. Second desktop client showing the same board and chat. These screenshots come from a local two-window test on one Windows computer.

Check	Result
TCP framing, UTF-8, malformed JSON and truncated frames	Passed
Concurrent edits: 4 clients, 40 drawing events	All clients received update numbers 1-40 in the same order
Room isolation, join/leave, chat and late joins	Passed
Host permission checks and saved-board recovery	Passed
Drawing tools, grouped undo, redo and duplicate requests	Passed
Identity recovery and server restart	Passed
Two GUI clients, mouse drawing, save/load, reconnect and SVG export	Passed

All 10 automated tests passed. The GUI check compared both clients' operation lists after edits and reconnect. The three PNGs in release/screenshots show the host, second client and board after loading the saved copy. Physical LAN latency and another computer's firewall have not been tested here.

Known limits

The server accepts up to 64 connections. Each room keeps 1000 drawing segments, 500 chat messages and one saved board. Saving the full JSON state after each edit is simple but can become slow with larger rooms. The file replacement does not guarantee recovery from sudden power loss. There is no offline editing, automatic host transfer or encrypted transport.